

Kubernetes and Cloud Native Associate - KCNA

Container Orchestration Service Mesh

Monoliths amp Microservices

When building one large application, a single failure can cause the entire system to break. In contrast, modifying smaller portions of your app helps isolate issues, thereby reducing risks during experimentation. This approach enables faster and more frequent deployments, fueling innovation and agility. Modern architectures favor smaller, independent components rather than relying on giant, interdependent systems.

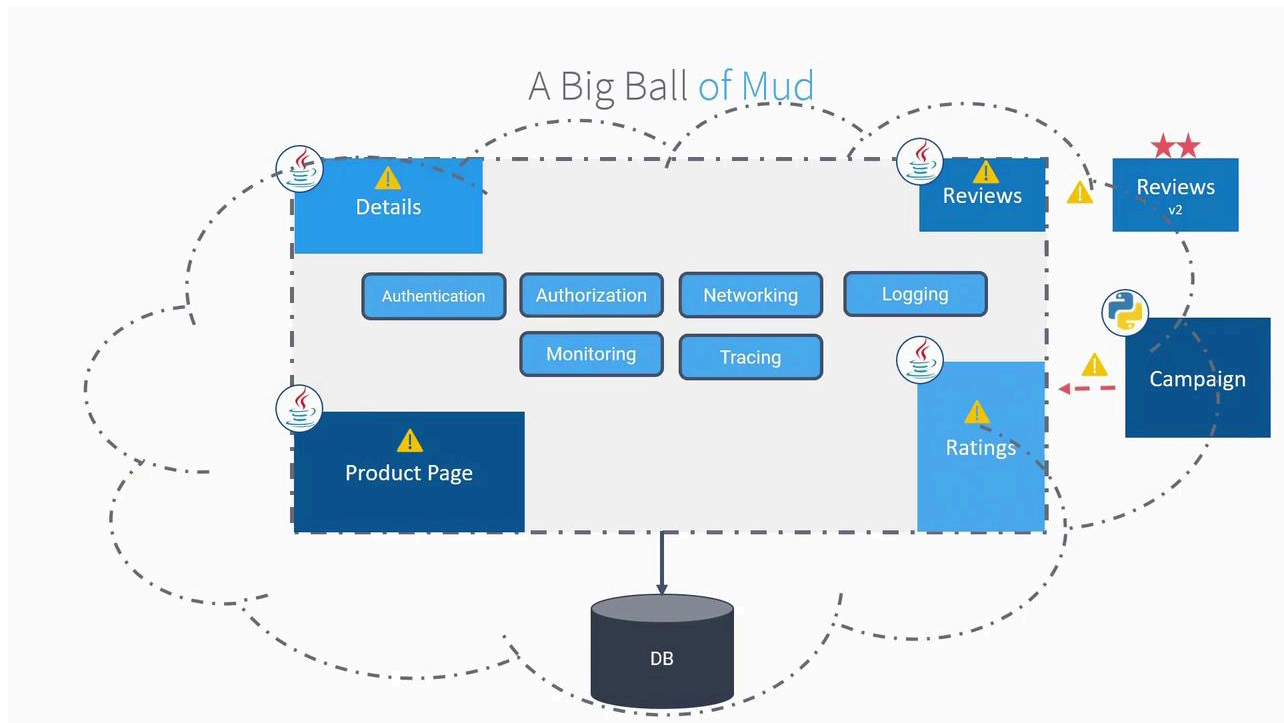
In traditional monolithic systems, all functionality is deployed together within a single code base, with no clear boundaries between features. These tightly coupled components typically run as a single process and share one database for persistence—a setup that can quickly become a performance bottleneck.

A Monolithic Book Info App



Consider a book information application as an example. This monolithic app comprises four modules: Details, Reviews, Ratings, and Product Page. Although the design appears modular, each service relies on a specific version of the others. This interdependency necessitates a full redeployment even when updating a single module. For instance, the product page aggregates data from the details and reviews modules, while the reviews module fetches rating data from the ratings module. All are implemented in Java and share common responsibilities such as networking, authentication, authorization, logging, monitoring, and data transfer. As a result, if the ratings module experiences issues due to heavy data loads, the entire system is affected. Adding new features—like a campaign module built in a different language or testing an updated version of the reviews module—requires redeploying the entire application because of the tightly intertwined architecture.

Over time, as applications accumulate decades of legacy code and involve numerous developers, the monolithic structure can degrade into what is commonly known as a "big ball of mud." This metaphor describes a tangled, loosely managed system with no clearly defined architectural boundaries.



Note

Transitioning from a monolithic model to a microservices architecture requires significant cultural, technical, and organizational changes. It is not an overnight process.

Transitioning to Microservices

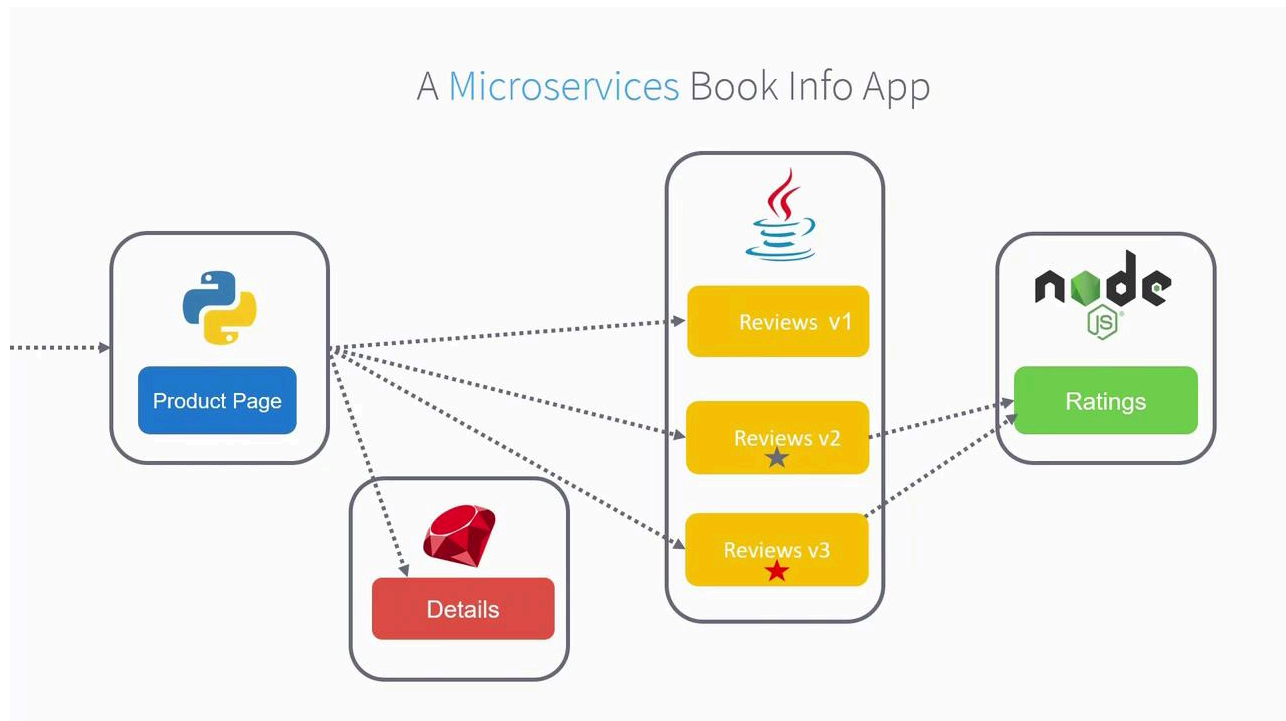
In a microservices architecture, each module is transformed into an independent application. For example:

- The product page is implemented as a Python application.
- The book details module is refactored into a Ruby application.
- The reviews module continues to run in Java.
- The ratings module is redesigned using Node.js.

Additionally, the reviews service is iterated with multiple versions (v1, v2, and v3) to test variations such as a no-star version (v1), a black star version (v2), and a red star

version (v3).

In the refactored system, users continue to access the product page, which now independently calls the details and reviews services. This decoupling allows each service to be individually scaled, upgraded, or fixed without impacting the others. The result is smaller, safer, and faster releases.



A significant advantage of this approach is that the ratings service can now scale dynamically based on customer load. Moreover, each service can be developed using a different programming language, granting teams greater autonomy and reducing the risk of a single point of failure. The independent nature of microservices enhances overall resilience by enabling isolated monitoring, updates, and rollbacks. In this transformed architecture, what was once a single monolithic application becomes a collection of six smaller, more manageable services.

In an ideal setup, each microservice maintains a single responsibility—a design principle that minimizes interdependencies and streamlines development.

Pros of Microservices



Scalability



Faster, smaller releases



**Technology and language agnostic
Development lifecycle**



System resiliency and isolation

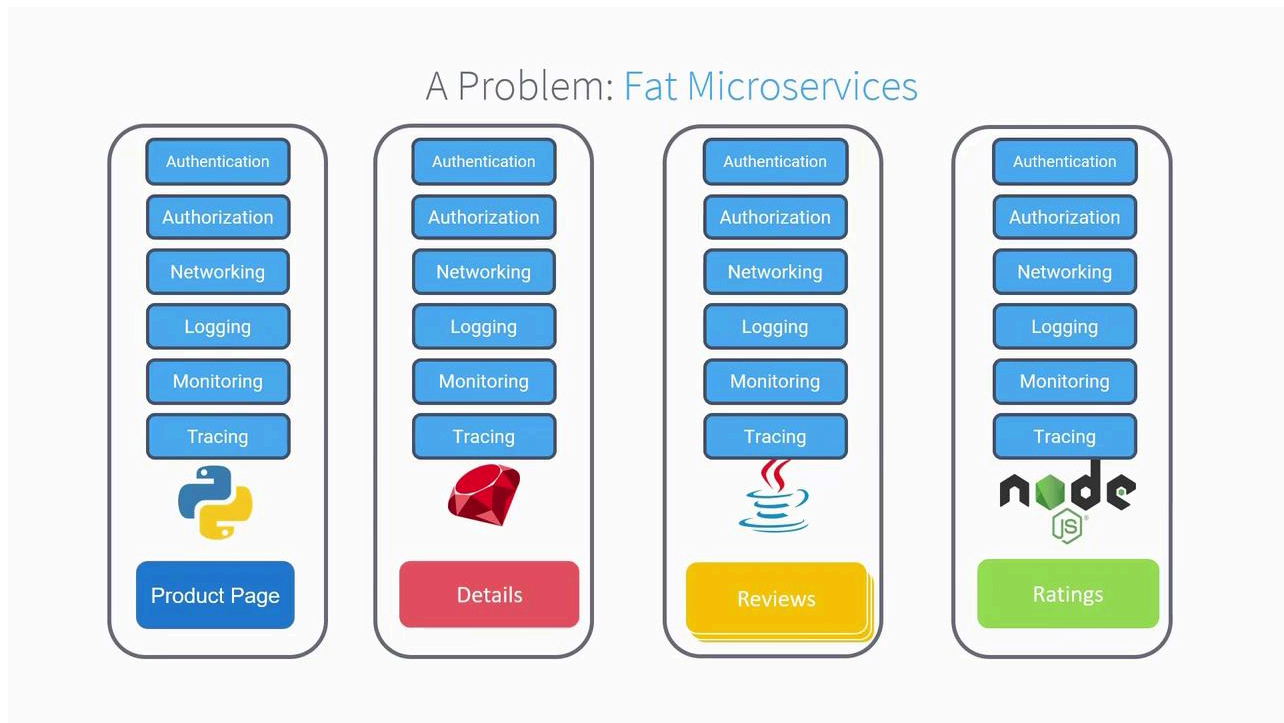


**Independent and easy
to understand services**

Challenges of Microservices

Transitioning to microservices also introduces new challenges. In a monolithic architecture, aspects such as networking, authentication, authorization, data transfer, logging, and monitoring were managed within a single code base. However, in a microservices environment, these concerns are often re-implemented in every service. This repetitive coding effort—known as cross-cutting concerns—can lead to code duplication and inconsistencies. For example, updating a certificate or a monitoring agent requires modifying each service individually.

This complexity is sometimes referred to as "fat microservices." Adding cross-cutting functionalities to each service can dilute the benefits of microservices by reintroducing interdependencies and making services heavier than intended.



Another challenge in microservices is managing inter-service communication. Determining which version of a service to call, handling traffic rules, and managing timeouts can be complex tasks. The increased number of services and abstraction layers often makes it more difficult to pinpoint issues, thereby necessitating a robust observability strategy.

Moreover, operating a microservices-based system—especially one spread across multiple programming languages and frameworks—can strain traditional operational practices. To tackle these issues, many organizations adopt DevOps practices, which promote closer collaboration between development and operations teams. This shared responsibility enables faster and more efficient deployment, monitoring, and maintenance of microservices.

Warning

Keep in mind that while microservices can improve scalability and resilience, they also introduce complexity in inter-service communication and require investment in developing robust monitoring and deployment practices.

In the upcoming lesson, we will explore how service meshes can simplify inter-service communication and enhance security within microservices architectures.



Beta



● Search docs...

⌘K

Kubernetes and Cloud Native Associate - KCNA ▾

- Service Mesh Explained

Tables can be used to compare different architectural elements:

Architecture Type	Characteristics	Example Issue
Monolithic	Single code base, tightly coupled components	Redeploying the full system for a minor update
Microservices	Independent services, isolated deployment	Managing cross-cutting concerns across services



Watch Video

Watch video content

Previous

Next

◀ Envoy

Service Mesh ▶